Nesterov Accelerated Gradient (NAG)

(Optimizers in Deep Learning – Part 3)

---

1. The Problem with Normal Momentum

We learned that **Momentum Optimizer** helps speed up convergence by remembering past gradients.

However, it also has a **drawback**:

Sometimes the momentum moves *too fast* and **overshoots the minimum** — especially when the optimizer is going downhill quickly and suddenly approaches the valley bottom.

Think of it like this:

- You're running downhill.
- You see the flat area (the minimum), but your speed is so high that you **overshoot** it and have to run back.

That's exactly what happens with **standard momentum** — it doesn't know *ahead of time* where it's heading.

---

2. The Solution - Nesterov Accelerated Gradient (NAG)

Nesterov Momentum (proposed by *Yurii Nesterov*, 1983) improves this by adding a **"look ahead" step**

Instead of:

Calculating the gradient at the *current* position

It does:

Calculates the gradient at the *projected (future)* position — where the momentum is about to take you.

This means NAG **anticipates the next step**, allowing it to **slow down early** if it's getting close to the minimum.

---

3. The Key Idea (Simplified Intuition)

Imagine:

- You're driving a car downhill (training the model).
- **Momentum** = pressing the accelerator (you keep moving fast).
- **NAG** = you *peek ahead* of the road — you see a curve coming and gently press the brake *before* reaching it.

So NAG is like a **smart driver** — it anticipates upcoming changes and adjusts speed smoothly.

---

4. Mathematical Formulation

Let's define the main components first:

Symbol	Meaning
(theta)	Model parameters (weights)
(v_t)	Velocity (momentum term)
(beta)	Momentum coefficient ($0 \leq \beta < 1$)
(eta)	Learning rate
(J(theta))	Loss function

Now, here's how NAG differs from regular momentum:

➢ Regular Momentum

$$v_t = \beta v_{t-1} + \eta \nabla_\theta J(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} - v_t$$

➢ Nesterov Accelerated Gradient (NAG)

$$v_t = \beta v_{t-1} + \eta \nabla_\theta J(\theta_{t-1} - \beta v_{t-1})$$

$$\theta_t = \theta_{t-1} - v_t$$

The difference:

The gradient is computed at **the "look-ahead" position**

$\rightarrow \theta_{t-1} - \beta v_{t-1}$,

not the current position θ_{t-1}.

This small change makes a **big improvement**.

---

5. Step-by-Step Working of NAG

Let's break it down clearly

1. **Look ahead:**

   Move temporarily in the direction of the previous velocity.

   $\rightarrow \theta_{\text{temp}} = \theta - \beta v_{t-1}$
2. **Compute gradient at this projected position:**

   $\rightarrow \nabla_\theta J(\theta_{\text{temp}})$
3. **Update the velocity:**

   $\rightarrow v_t = \beta v_{t-1} + \eta \nabla_\theta J(\theta_{\text{temp}})$
4. **Update the actual weights:**

   $\rightarrow \theta = \theta - v_t$

This "look-ahead" trick helps the optimizer see what's coming and adjust more wisely.

---

6. Geometric Intuition (Visual Understanding)

- Imagine a **curved valley** again.
- **Momentum** keeps moving based on past direction — often overshooting the bottom.
- **NAG**, however, first *checks ahead* where it would land if it moved with full momentum.
- If the slope ahead is shallow (approaching minimum), it **slows down early**.
- This creates a **more stable and controlled** approach to the minimum.

In short:

NAG = Momentum + Anticipation

---

7. Mathematical Intuition (Why It Works Better)

When gradients are steep:

- Momentum (βv) increases → faster movement.

When gradients are small near the minimum:

- The look-ahead gradient is smaller → velocity automatically decreases.

So NAG automatically **reduces overshooting** and **smooths convergence** near the optimal point.

---

8. Keras Implementation Example

In Keras or TensorFlow, you can use NAG directly by enabling the parameter nesterov=True.

```
from tensorflow.keras.optimizers import SGD

optimizer = SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
```

It adds the NAG logic internally — computing gradients at the look-ahead position.

---

9. Disadvantages of NAG

While NAG is powerful, it also has a few limitations:

1. **More Computation per Step**

- Since gradients are computed at the *look-ahead* position, it's a bit slower per iteration.

2. **Still Needs Tuning**

- Learning rate (η) and momentum (β) still need to be tuned carefully.

3. **Less Intuitive at First**

- For beginners, it's harder to visualize than standard momentum.

---

10. Typical Hyperparameter Settings

Parameter	Common Value
Learning rate (η)	0.01 or 0.001
Momentum (β)	0.9
Use Nesterov?	True

These settings often work well for most networks (especially CNNs).

---

11. Summary Table — Momentum vs NAG

Feature	Momentum	Nesterov Accelerated Gradient
Gradient Computed At	Current position	Look-ahead position
Overshooting	Possible	Reduced
Convergence Speed	Fast	Faster & more stable
Behavior Near Minima	May oscillate	Smooth approach
Implementation	Easier	Slightly complex
Supported In Keras?	✅ Yes	✅ Yes (with nesterov=True)

---

12. Key Takeaways

1. NAG = Momentum with foresight
2. Reduces zig-zag motion and overshooting
3. Makes convergence faster and smoother
4. Computes gradient at a projected point ahead
5. Supported by most deep learning frameworks

TL;DR (In One Line)

Nesterov Accelerated Gradient (NAG) improves on momentum by "looking ahead" before updating allowing the model to move faster yet slow down intelligently near the optimum.
